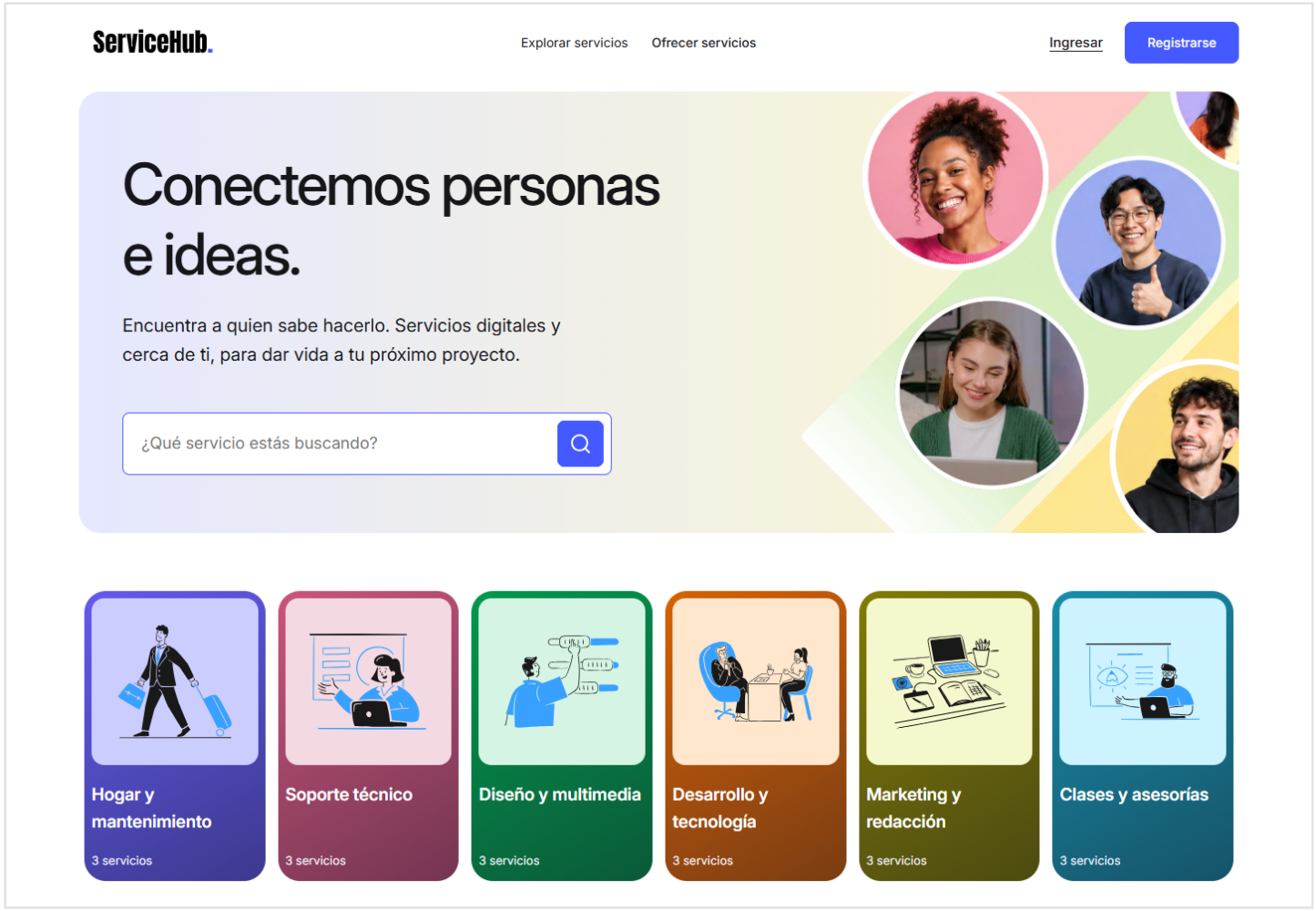
Conectemos personas e ideas.

Primera implementación integrada de un marketplace de servicios en Perú



Home real en Edge, 1440 px. Recorte superior: hero y seis categorías.

El producto

Servicios digitales y presenciales, perfiles profesionales, conversación, oferta, contratación y seguimiento.

Una cuenta puede contratar y ofrecer. Todos los ejemplos son ficticios; los pagos son simulados.

Base evaluable

- Angular 20.3.33 / TypeScript 5.9.3
- Bootstrap 5.3.8 personalizado
- JSON Server 0.17.4 + db.json
- Figma, Inter local y assets originales

Estructura y servicios REST

La vista navega con Angular Router; los recursos persisten con el router de JSON Server

Organización del proyecto

core/

HttpClient + Observable, sesión y guards.

layouts/ y shared/

Shell, tema, ServiceCard, diálogo, pipe y directiva.

features/

Catálogo, perfiles, publicación, mensajes, listas y pedidos.

shared/models.ts

Contratos TypeScript de las entidades.

server/

Validación, permisos, archivos y operaciones compuestas.

db.json y uploads/

Datos e imágenes persistentes. No se resetean al iniciar.

Navegador → /api → JSON Server

El proxy de Angular conecta 4200 con 3000. Las colecciones usan el router original; el frontend no importa db.json ni simula cambios con localStorage.

- GET: catálogo, perfiles, mensajes y tablas.
- POST: servicios, mensajes y listas.
- PATCH: editar/pausar servicio y responder reseña.
- DELETE: borradores elegibles y listas propias.

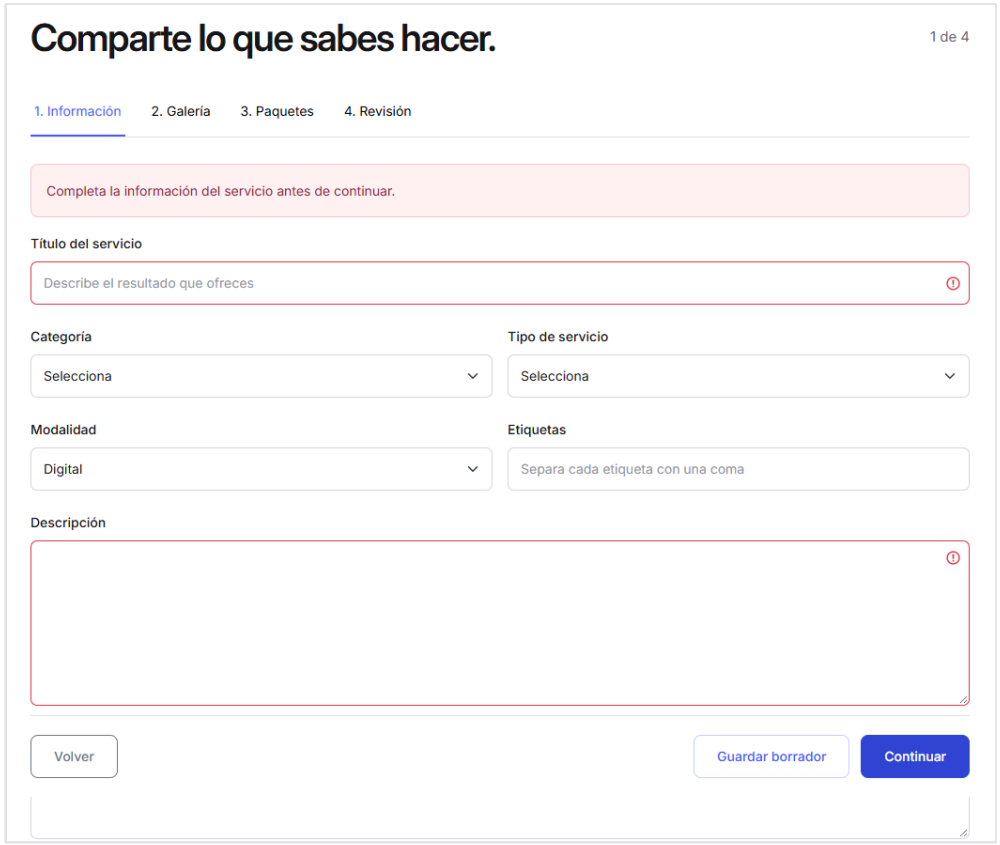
Paginación comprobada

GET /api/services?_page=2&_limit=8
Array de 8 registros. X-Total-Count: 18 y header Link.

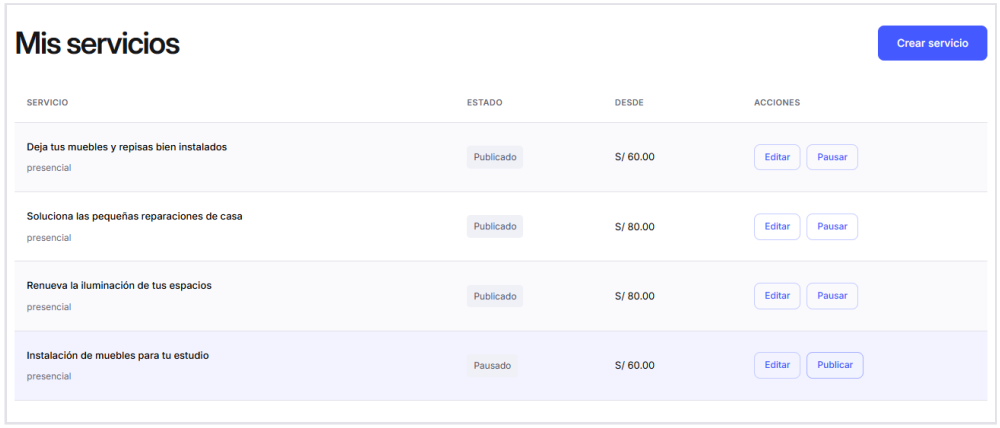
Acciones de checkout: sesión, importe calculado por servidor, snapshot, una escritura lógica y reintento idempotente. No se atribuyen transacciones ACID a JSON Server.

Formularios, CRUD y validación

Reactive Forms, alertas con NgIf y tablas Bootstrap conectadas a persistencia real



Recorte de formulario inválido: mensaje y campos señalados; barra inferior fija.



Recorte de la tabla tras crear, publicar, pausar y recargar un servicio.

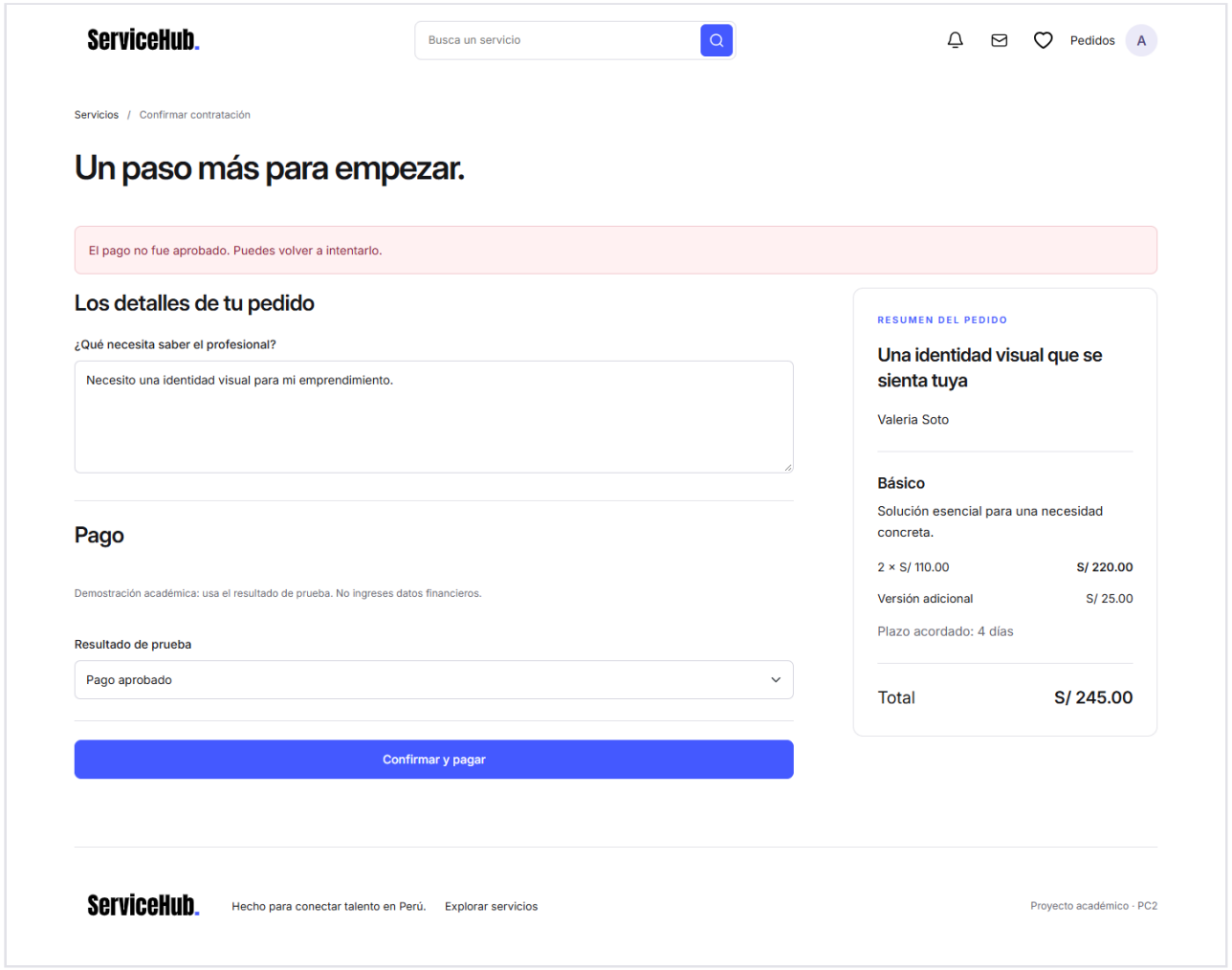
Verificado en API y navegador

- GET/POST/PATCH/DELETE con propiedad validada.
- Un cliente incompleto no puede publicar.
- Cambios persisten tras recargar y reabrir la base.
- Clases btn, form-control y table-striped/table-hover.

Capturas completas: 07-formulario-validacion.png, 08-formulario-paquetes.png y 09-mis-servicios-crud.png.

Del paquete al pedido

Pipe precioSoles visible, extras, pago simulado y snapshot del acuerdo



Checkout real de la prueba: cantidad 2, extra y total mostrado en soles.

S/ 245.00

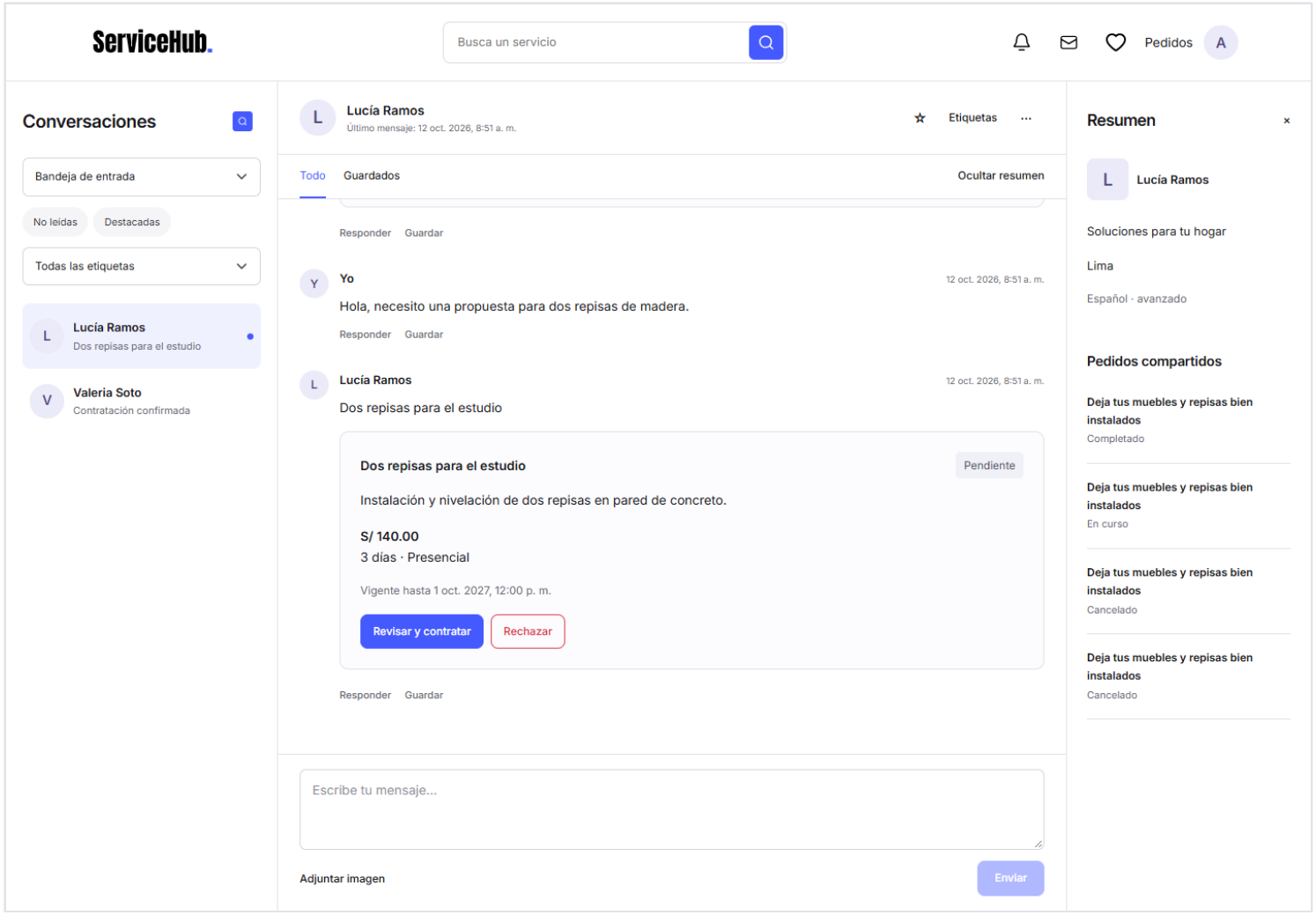
11000 × 2 + 2500
= 24500 céntimos

El pipe recibe céntimos y divide una sola vez.
El servidor recalcula antes de confirmar.

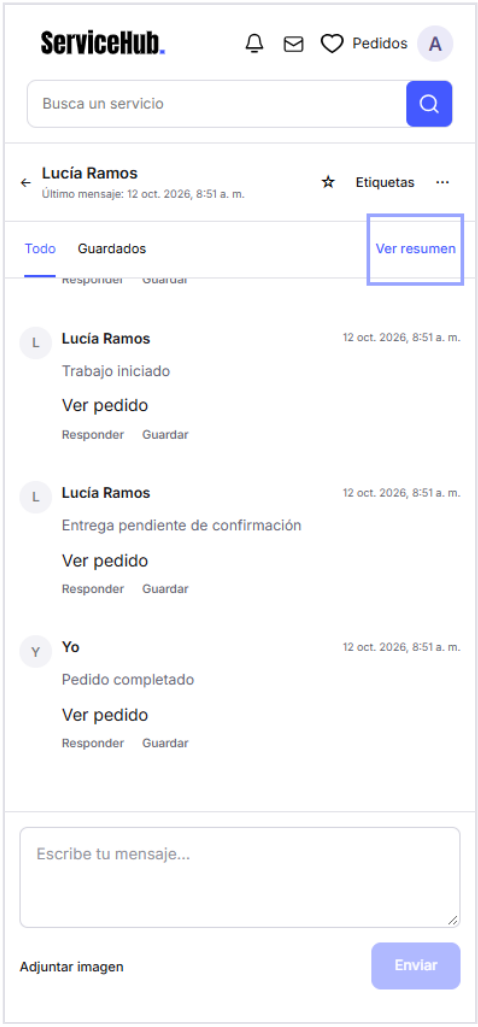
- Pago fallido: no crea pedido.
- Reintento: retorna el acuerdo existente.
- Oferta: máximo un pedido.
- Inicio → entrega → ajuste o confirmación.
- Reseña tras completar o cancelar después del inicio.

Conversaciones y responsive

Dos perfiles de navegador coordinan una oferta y completan una contratación



Edge 1440 px: mensaje, oferta real de S/ 140.00 y resumen lateral.



390 px: chat y acciones legibles; resumen en drawer con Escape.

Resultados y límites de la entrega

Evidencia reproducible; la primera versión académica conserva pendientes explícitos

Verificación ejecutada

- Instalación limpia desde lockfile: correcta.
- Build correcto: 600.83 kB inicial / 117.88 kB estimados de transferencia.
- 5 de 5 unitarios (Vitest 4.1.11).
- 14 de 14 pruebas de API; bases temporales.
- 6 de 6 recorridos E2E en Edge.
- 18 capturas reales: desktop, 390 y 768 px.
- Reinicio preserva pedidos, mensajes e imágenes; sesión nueva al volver a ingresar.

Instalación final: 0 vulnerabilidades conocidas reportadas. Se corrigieron versiones y overrides y se repitieron las verificaciones.

Pendientes identificados

- Reducir CSS inicial: warning de 100.83 kB sobre presupuesto de 500 kB.
- Filtros por día local Lima; actualmente comparan fecha UTC.
- Revisión de accesibilidad/360 px y refinamiento de formularios.
- Portadas reales, trayectoria estructurada y reputación agregada en perfil.
- Retención y decodificación completa de imágenes; separar efectos secundarios de CRUD.

Demostración

npm ci · npm start · http://127.0.0.1:4200
cliente@servicehub.test / profesional@servicehub.test
Contraseña: ServiceHub123!

Consultar README, matriz-cumplimiento.md y docs/estado-actual.md. No hay pagos reales, despliegue público ni transacciones ACID. Sin commit/push del agente.